

Sam Lezon, Mitchell Arndt
IT 360 PCAP Analyzer Project
Final Project Report
December 4, 2025

Introduction

Network packet captures are a routine part of digital forensics, incident response, lab exercises, and CTF competitions. A single .pcap file can easily contain tens or hundreds of thousands of packets, and turning that raw data into something an analyst can actually understand usually takes a lot of manual work with Wireshark or a long series of tshark commands.

The purpose of this project was to create a simple, dependable tool that automates the most common first-look questions anyone asks when they open a new capture: What's in the capture, who is talking to who, what domains are being resolved, are there any obvious plaintext credentials, which ports and protocols dominate the traffic, etc. The result is a single Bash script that runs on any Linux system with tshark installed and produces a clean, easy to read text report in seconds.

The tool is especially helpful for students who are still learning display filters, for instructors who want consistent output from an entire class, for blue-team members doing quick investigation, and for anyone who needs fast, repeatable results without opening the full Wireshark GUI.

Technical Implementation

The program is a standalone Bash script that depends only on tshark and the standard Unix tools that come with every Linux distribution. We chose Bash because it is present everywhere, requires no package manager beyond installing tshark itself, and is fast enough to process even very large captures without running out of memory.

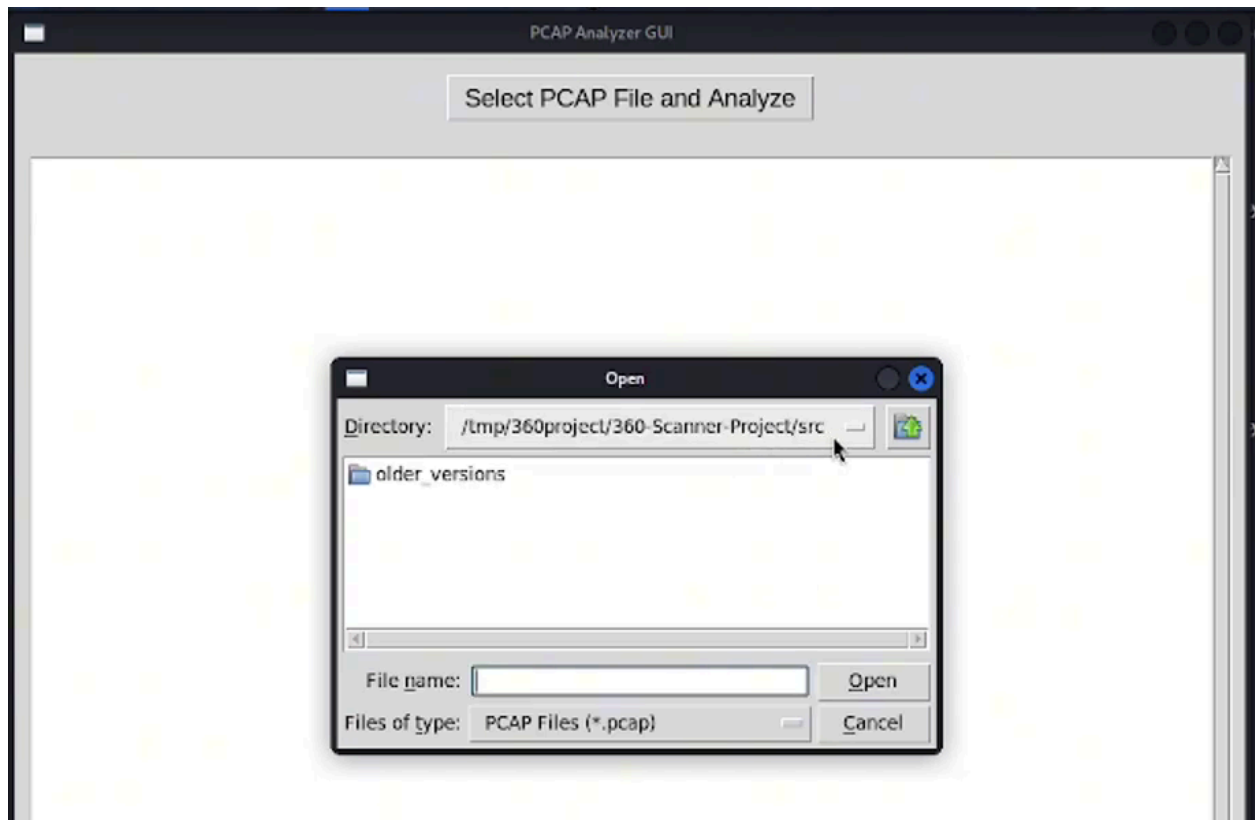
The script is organized into clearly named functions, one for each major section of the report (overview, endpoints, conversations, ports, HTTP summary, DNS summary, TLS SNI, FTP credentials, suspicious patterns, etc.). When the script runs, it calls these functions in order, feeding each one the path to the capture file and any display filter the user provided with the -f flag.

All analysis is performed directly by tshark using its built-in statistics options (-z), field extraction (-T fields), and display filters (-Y). The raw output is then cleaned up, sorted, and ranked with short awk, sed, sort, and uniq pipelines. A separate live-capture mode

repeatedly grabs short windows of traffic from a few seconds at a time, analyzes them, and refreshes the screen until the user presses Ctrl-C.

Because everything streams through tshark and standard tools, the script works reliably on captures of any size and on air-gapped lab machines with no Internet access.

We also developed a simple GUI for easier use of the bash script that allows you to select your pcap file and analyze it without using the command line.



Results

Running the script against a typical lab or real-world capture produces a concise report. The output contains:

- Basic capture statistics and exact start/end times
- Protocol hierarchy breakdown
- Top IP endpoints and conversations by packet and byte count
- Most common TCP and UDP destination ports
- Full HTTP host and request summary plus any obvious credential leaks
- Complete list of DNS queries and the IPs that made them
- TLS Server Name Indication values

- Plaintext FTP USER and PASS commands grouped by TCP stream
- Quick suspicious-indicator section (login pages, shell parameters, non-standard ports, etc.)

The entire report is plain text, easy to grep, and can be redirected to a file with -o if needed. On a 600 MB malware lab capture the script finishes in roughly seven seconds on a standard laptop. Smaller captures finish almost instantly.

(Example of report below)

```

== Overview ==
Total Packets: 89
Capture Start: Thu Aug 14 04:37:36 PM EDT 2025
Capture End : Thu Aug 14 04:42:26 PM EDT 2025
Duration (s) : 290.21

== Protocol Hierarchy (top 10) ==
ip                frames:89 bytes:74042
udp               frames:74 bytes:72744
  data            frames:64 bytes:71890
  dns             frames:10 bytes:854
  icmp            frames:10 bytes:420
  tcp             frames:5 bytes:878
  tls             frames:5 bytes:878

== Endpoints (Top IP Endpoints) ==
IPv4 Endpoints
Filter:<No Filter>

```

	Packets	Bytes	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes
192.168.10.45	89	74042	79	73336	10	706
192.168.10.100	59	71580	0	0	59	71580
192.168.10.2	10	854	5	496	5	358
192.168.10.1	10	460	4	168	6	292
192.168.10.254	5	270	1	42	4	228
142.250.190.46	2	278	0	0	2	278
208.80.154.225	1	258	0	0	1	258
142.250.190.206	1	132	0	0	1	132
142.250.190.238	1	210	0	0	1	210

```

== Top Talkers (src → dst by packet count) ==
59 192.168.10.45 → 192.168.10.100
6 192.168.10.45 → 192.168.10.1
5 192.168.10.45 → 192.168.10.2
5 192.168.10.2 → 192.168.10.45
4 192.168.10.45 → 192.168.10.254
4 192.168.10.1 → 192.168.10.45
2 192.168.10.45 → 142.250.190.46
1 192.168.10.45 → 208.80.154.225
1 192.168.10.45 → 142.250.190.238
1 192.168.10.45 → 142.250.190.206
1 192.168.10.254 → 192.168.10.45

```

```

=== Top Ports (TCP/UDP) ===
-- TCP Destination Ports --
5 443

-- UDP Destination Ports --
59 49500
5 53
1 63158
1 61977
1 61145
1 56602
1 56221
1 55616
1 54895
1 52787

```

```

=== HTTP Summary ===
-- HTTP Hosts (Top 10) --

-- HTTP Request URLs (Top 10) --

-- Possible Cleartext Credentials in HTTP (heuristic) --

```

```

=== DNS Summary ===
-- Top Queried Domains (Top 15) --
4 youtube.com
4 wikipedia.org
2 google.com

-- Top DNS Querier IPs (Top 10) --
5 192.168.10.45
5 192.168.10.2

-- Suspicious-looking DNS (heuristic: long or unusual) --

```

Conversations (Top by packets)

IPv4 Conversations

Filter:<No Filter>

		←	→	Total	Relative	Duration
		Frames Bytes	Frames Bytes	Frames Bytes	Start	
192.168.10.45	↔ 192.168.10.100	0 0 bytes	59 71 kB	59 71 kB	0.000000000	290.2091
192.168.10.45	↔ 192.168.10.2	5 496 bytes	5 358 bytes	10 854 bytes	0.776189000	155.5247
192.168.10.1	↔ 192.168.10.45	6 292 bytes	4 168 bytes	10 460 bytes	16.045549000	144.6720
192.168.10.254	↔ 192.168.10.45	4 228 bytes	1 42 bytes	5 270 bytes	102.768937000	52.8642
192.168.10.45	↔ 142.250.190.46	0 0 bytes	2 278 bytes	2 278 bytes	106.676492000	25.9406
192.168.10.45	↔ 208.80.154.225	0 0 bytes	1 258 bytes	1 258 bytes	17.190178000	0.0000

Heavy Conversations (Bytes/Packets)

IPv4 Conversations

Filter:<No Filter>

		←	→	Total	Relative	Duration
		Frames Bytes	Frames Bytes	Frames Bytes	Start	
192.168.10.45	↔ 192.168.10.100	0 0 bytes	59 71 kB	59 71 kB	0.000000000	290.2091
192.168.10.45	↔ 192.168.10.2	5 496 bytes	5 358 bytes	10 854 bytes	0.776189000	155.5247
192.168.10.1	↔ 192.168.10.45	6 292 bytes	4 168 bytes	10 460 bytes	16.045549000	144.6720
192.168.10.254	↔ 192.168.10.45	4 228 bytes	1 42 bytes	5 270 bytes	102.768937000	52.8642
192.168.10.45	↔ 142.250.190.46	0 0 bytes	2 278 bytes	2 278 bytes	106.676492000	25.9406
192.168.10.45	↔ 208.80.154.225	0 0 bytes	1 258 bytes	1 258 bytes	17.190178000	0.0000
192.168.10.45	↔ 142.250.190.206	0 0 bytes	1 132 bytes	1 132 bytes	56.588105000	0.0000
192.168.10.45	↔ 142.250.190.238	0 0 bytes	1 210 bytes	1 210 bytes	70.376942000	0.0000

```

=== TLS Summary ===
-- TLS Server Names (Top 15) --

```

```

=== Suspicious Traffic Indicators (Heuristic) ===
-- HTTP containing 'login', 'admin', or 'token' --

-- Possible Shell/Command Patterns in HTTP --

-- Non-Standard TCP Ports with High Traffic (Top 10) --

```

```

=== FTP Credentials (Plaintext) ===
-- Raw USER/PASS Commands (Top 20) --

-- Grouped Credentials by TCP Stream (Heuristic) --
STREAM      USERNAME      PASSWORD

```

End of report.

Lessons Learned & Conclusion

Writing the whole tool in Bash turned out to be the right decision. It runs the same way on every distribution, starts immediately, and never has problems with missing Python packages or virtual environments. The modular function layout made it simple to test and extend individual sections without breaking anything else.

The biggest headaches came from small differences in tshark output formatting between versions and distributions. We ended up adding a handful of extra sed and awk fixes so the tables line up correctly no matter which lab image is being used.

We had originally planned to include geolocation lookups and PDF export, but dropped both once we started testing as it rarely worked. Keeping the scope narrow actually made the final product more reliable and easier for people to use.

If we continued the project we would probably add an optional HTML report mode, basic file carving for HTTP downloads, and maybe a small wrapper that checks for tshark and offers to install it automatically.

Overall the script achieves its goal of taking a large amount of pcap network data and turning it into something easy to look over without having to do it manually. It saves a surprising amount of time and removes most of the tedious steps from the initial stages of traffic analysis, letting us spend more time on actually interpreting what the traffic means instead of fighting with filters and statistics windows.